

NLP Group Assignment 1

Full Comparative Report — Questions 1–4

Group Members

Agastya Nath (12340140)
Paritosh Lahre (12341550)
Y. Rahul Dev Reddy (12342390)
Varun Rao (12342320)

September 10, 2026

Contents

Preface **3**

1 Question 1: Word Segmentation and POS Tagging **4**
1.1 Objective . . . 4
1.2 Approach . . . 4
1.3 Data handling . . . 4
1.4 How to reproduce . . . 5
1.5 Results summary . . . 5
1.5.1 POS accuracy . . . 5
1.5.2 Improvement over baselines . . . 5
1.5.3 Error-source breakdown . . . 6
1.6 Sample outputs . . . 6
1.6.1 English 1 — `thequickbrownfoxjumpsoverthelazydog` . . . 6
1.6.2 English 2 (own) — `tobeornottobethatisthequestion` . . . 7
1.6.3 Spanish 1 — `mispadrespuedenviajar` . . . 7
1.6.4 Spanish 2 — `elcielodespejadoesazul` . . . 7
1.6.5 Spanish 3 (own) — `maríaleeunlibro` . . . 7
1.7 Answers to the assignment questions . . . 8
1.7.1 Where did English and Spanish differ most in accuracy? . . . 8
1.7.2 Did agreement-aware tagging help, or add noise? . . . 8
1.7.3 How much error came from segmentation vs. genuine tagging? . . . 8
1.7.4 How much better were the models than the simple baselines? . . . 9
1.8 Q1 conclusion . . . 9

2 Question 2: Transition-Based Dependency Parser **10**
2.1 Objective . . . 10
2.2 Design choices . . . 10
2.2.1 Arc-standard parser state . . . 10
2.2.2 Static gold oracle . . . 11
2.3 How training output is generated . . . 11
2.3.1 Step 1 — Read the treebank . . . 11
2.3.2 Step 2 — Generate oracle instances . . . 11
2.3.3 Step 3 — Extract and encode features . . . 11
2.3.4 Step 4 — Train and save the classifier . . . 12
2.4 Parsing and example output . . . 12
2.5 LAS evaluation . . . 12
2.6 Reproduce the final run . . . 12
2.7 Final results . . . 13

3 Question 3: Spelling Corrector **15**

3.1 Objective . . . 15
3.2 Part 1 — Corpus and model preparation . . . 15
3.3 Part 2 — Candidate generation . . . 16
3.3.1 Method A — standard edit-distance-1 . . . 16
3.3.2 Method B — symmetric delete (SymSpell) . . . 16
3.4 Part 3 — Correction logic . . . 16
3.4.1 Non-word errors . . . 16
3.4.2 Real-word errors . . . 16
3.4.3 Why interpolated smoothing for the decision . . . 16
3.5 Part 4 — Evaluation . . . 17
3.5.1 Test-set construction . . . 17
3.5.2 Accuracy . . . 17
3.5.3 Threshold trade-off . . . 18
3.5.4 Residual errors (systematic) . . . 19
3.5.5 Speed Demon benchmark . . . 19
3.6 Part 5 — Live interactive application . . . 20
3.6.1 Brief test sentences . . . 21
3.7 Q3 summary . . . 21

4 Question 4: Live Background Editor **22**
4.1 Objective . . . 22
4.2 Configuration choices . . . 22
4.3 Tagset reconciliation . . . 22
4.4 Decision rule (Part 4) . . . 23
4.5 Sample run A (`-seed 42`, bible-kjv) . . . 23
4.6 Sample run B (`-seed 7`, chesterton-thursday) . . . 23
4.7 Speed Demon (1000 words) . . . 24
4.8 Comparative analysis . . . 24
4.9 Known limitations . . . 25
4.10 Streamlit (Part 5) . . . 25
4.11 How to reproduce . . . 25

5 Overall Conclusion **26**

Preface

This document consolidates the comparative reports for **Questions 1–4** of the NLP Group Assignment. Each chapter mirrors the corresponding implementation folder in the group repository and the numbers reported from evaluation runs (`evaluation.json`, `results.json`, `demo_outputs.json`, etc.).

Q	Folder	Topic
1	`q1/`	Word segmentation + POS tagging (EN + ES)
2	`q2/`	Transition-based dependency parser
3	`q3/`	Spelling corrector (non-word + real-word)
4	`q4/`	Live background editor (Streamlit)

Chapter 1

Question 1: Word Segmentation and POS Tagging

1.1 Objective

Given a sentence with all spaces removed, the system must:

1. **Segment** the string into words;
2. **POS-tag** each word.

We compare **English** (Brown) and **Spanish** (UD Spanish-GSD), and evaluate morphology-aware tags for Spanish agreement (gender/number).

1.2 Approach

- **Segmentation:** trigram language model + Viterbi dynamic programming.
- **POS tagging:** trigram HMM (emission + transition) + Viterbi.
- **Morphology-aware Spanish tags:** UPOS extended with Gender/Number where available (e.g. `NOUN-Masc-Pl`).
- **Baselines:** greedy longest-match segmentation; most-frequent-tag POS.

1.3 Data handling

Table 1.1: Corpora and splits.

Language	**Corpus**	**Train / Dev / Test**
English	NLTK Brown tagged sentences	80/20 sequential (45,872 / 11,468)
Spanish	UD Spanish-GSD CoNLL-U	Official (14,186 / 1,400 / 427)

Segmentation LMs use **lowercased** forms to match no-space evaluation strings. POS taggers keep corpus tagsets: **Brown** for English (`AT`, `NN`, `JJ`, ...) and **UPOS** / morph tags for Spanish. (The assignment PDF illustration uses Penn-style `DT`/`NN`; Brown tags are correct for this corpus.)

1.4 How to reproduce

```
cd q1
mkdir -p data/spanish
git clone https://github.com/UniversalDependencies/UD_Spanish-GSD.git \
  data/spanish/UD_Spanish-GSD
python main.py train-english
python main.py sample
python main.py evaluate   # -> results/evaluation.json
```

1.5 Results summary

1.5.1 POS accuracy

- English Viterbi POS (gold segmentation): **92.94%** (exact 92.9390%).
- Spanish plain Viterbi POS: **92.94%** (exact 92.9417%).

Typical English confusions: `NN`↔`JJ`/`VB`/`NP`. Typical Spanish confusions: `NOUN`↔`PROPN`/`ADJ`, `VERB`↔`AUX`.

```
Viterbi POS accuracy (subset):        92.6035%
Most-frequent-tag accuracy (subset):  87.4005%
(Report tables still use full-run numbers from evaluation.json)

Confusion Matrix
----------------
Actual \ Predicted  NN    IN    .     AT    ,     RB    JJ    VBD   VB    CC    NNS   NP    PPSS  PPS   CS
NN                  1444  1     3     1     3     0     27    1     11    1     7     4     1     1     0
IN                  1     1133  0     0     0     4     1     0     0     3     0     0     0     0     1
.                   0     0     1122  0     0     0     0     0     0     0     0     0     0     0     0
AT                  0     0     0     1023  0     3     0     0     0     0     0     0     0     0     0
,                   0     0     0     0     825   0     0     0     0     0     0     0     0     0     0
RB                  8     11    1     1     1     589   17    0     0     5     1     0     0     0     2
JJ                  10    1     1     6     1     13    567   1     0     0     1     3     0     0     0
VBD                 6     1     0     0     3     0     1     513   3     0     0     0     0     0     0
VB                  19    0     0     0     0     3     2     2     534   0     0     1     0     0     1
CC                  0     0     0     0     0     0     0     0     0     463   0     0     0     0     0
NNS                 13    1     0     0     0     1     3     0     0     0     376   0     0     0     0
NP                  20    1     1     0     4     13    11    0     1     1     2     238   3     52    0
PPSS                1     0     0     0     0     0     0     0     0     0     0     0     373   0     0
PPS                 0     0     0     0     0     0     0     0     0     0     0     0     0     375   0
CS                  2     15    0     0     0     4     1     0     0     0     0     0     0     0     314
```

Figure 1.1: English POS confusion matrix and accuracy (terminal output from evaluation / screenshots run).

1.5.2 Improvement over baselines

Table 1.2: Model vs. baseline.

Task	Lang.	Model	Baseline	Δ
Segmentation	English	48.85%	25.50%	**+23.35 pp**
Segmentation	Spanish	14.99%	6.09%	**+8.90 pp**
POS tagging	English	92.94%	87.29%	**+5.65 pp**
POS tagging	Spanish	92.94%	88.55%	**+4.39 pp**
Morph POS	Spanish	90.57%	—	**−2.38 pp** vs plain

Exact values from `evaluation.json`: EN seg 48.8490% vs 25.4970% (+23.3519); ES seg 14.9883% vs 6.0890% (+8.8993); EN POS 92.9390% vs 87.2853% (+5.6537); ES POS 92.9417% vs 88.5500% (+4.3917); morph 90.5667% (−2.3750 vs plain).

1.5.3 Error-source breakdown

Table 1.3: Segmentation-caused vs. genuine tagging errors (segment-then-tag pipeline).

Lang.	Seg-caused	Genuine	Share	Words
English	16,305	8,405	65.99% / 34.01%	181,546
Spanish	2,415	506	82.68% / 17.32%	12,000

English total identified errors: 24,710. Segmentation accounts for nearly **two-thirds** of English pipeline errors and over **four-fifths** of Spanish ones.

```
English
  Total gold words: 181546
  Sentences analyzed: 11468
  Segmentation-caused errors: 16305
  Genuine tagging errors: 8405
  Segmentation-caused: 65.99%
  Genuine tagging: 34.01%

Spanish
  Total gold words: 12000
  Sentences analyzed: 427
  Segmentation-caused errors: 2415
  Genuine tagging errors: 506
  Segmentation-caused: 82.68%
  Genuine tagging: 17.32%
```

Figure 1.2: Error-source analysis: segmentation-caused vs. genuine tagging mistakes.

1.6 Sample outputs

Produced by Viterbi **segment-then-tag**. Morph tags shown for Spanish.

1.6.1 English 1 — `thequickbrownfoxjumpsoverthelazydog`

```
Segmentation: ['the', 'quick', 'brown', 'fox', 'jumps', 'overt', 'he', 'lazy', 'dog']
POS: [('the','AT'),('quick','JJ'),('brown','JJ'),('fox','NN'),('jumps','VBZ'),
      ('overt','JJ'),('he','PPS'),('lazy','JJ'),('dog','NN')]
```

Note: *over the* is mis-segmented as *overt*|*he* — a typical LM preference failure when both paths are in-vocabulary.

1.6.2 English 2 (own) — `tobeornottobethatisthequestion`

```
Segmentation: ['to', 'be', 'or', 'nott', 'obe', 'that', 'is', 'the', 'question']
POS: [('to','TO'),('be','BE'),('or','CC'),('nott','*'),('obe','NP'),
      ('that','WPS'),('is','BEZ'),('the','AT'),('question','NN')]
```

1.6.3 Spanish 1 — `mispadrespuedenviajar`

```
Segmentation: ['mis', 'padres', 'pueden', 'viajar']
POS:   [('mis','DET'),('padres','NOUN'),('pueden','AUX'),('viajar','VERB')]
Morph: [('mis','DET-Pl'),('padres','NOUN-Masc-Pl'),('pueden','AUX-Pl'),('viajar','VERB
    ')]
```

1.6.4 Spanish 2 — `elcielodespejadoesazul`

```
Segmentation: ['el', 'cielo', 'des', 'pe', 'j', 'ado', 'es', 'azul']
POS:   [('el','DET'),('cielo','NOUN'),('des','ADP'),('pe','PROPN'),
        ('j','PROPN'),('ado','PROPN'),('es','AUX'),('azul','ADJ')]
Morph: [('el','DET-Masc-Sg'),('cielo','NOUN-Masc-Sg'),('des','ADP'),('pe','PROPN'),
        ('j','PROPN'),('ado','PROPN-Masc-Sg'),('es','AUX-Sg'),('azul','ADJ-Sg')]
```

despejado is over-segmented — illustrating Spanish boundary difficulty.

1.6.5 Spanish 3 (own) — `maríaleeunlibro`

```
Segmentation: ['mara', 'lee', 'un', 'libro']
POS:   [('mara','PROPN'),('lee','VERB'),('un','DET'),('libro','NOUN')]
Morph: [('mara','PROPN'),('lee','VERB-Sg'),('un','DET-Masc-Sg'),('libro','NOUN-Masc-Sg
    ')]
```

Also: `lacasarojaesgrande` → `la/DET casa/NOUN roja/ADJ es/AUX grande/ADJ` (morph shows Fem-Sg agreement on *la*/*casa*/*roja*).

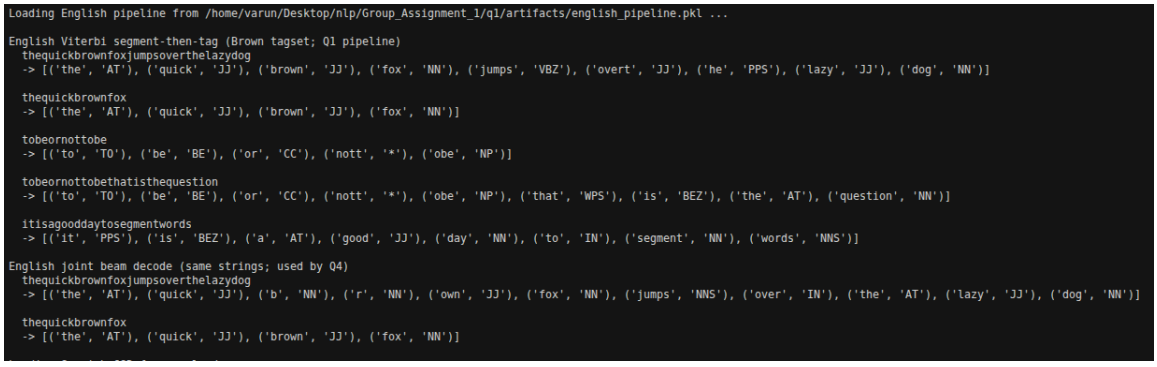


Figure 1.3: Sample segment-then-tag outputs (part A).

```
Loading Spanish-GSD for sample demos ...

Spanish segment + plain POS / morph POS
  mispadrespuedenviajar
  words -> ['mis', 'padres', 'pueden', 'viajar']
  plain -> [('mis', 'DET'), ('padres', 'NOUN'), ('pueden', 'AUX'), ('viajar', 'VERB')]
  morph -> [('mis', 'DET-Pl'), ('padres', 'NOUN-Masc-Pl'), ('pueden', 'AUX-Pl'), ('viajar', 'VERB')]

  elcielodespejadoesazul
  words -> ['el', 'cielo', 'des', 'pe', 'j', 'ado', 'es', 'azul']
  plain -> [('el', 'DET'), ('cielo', 'NOUN'), ('des', 'ADP'), ('pe', 'PROPN'), ('j', 'PROPN'), ('ado', 'PROPN'), ('es', 'AUX'), ('azul', 'ADJ')]
  morph -> [('el', 'DET-Masc-Sg'), ('cielo', 'NOUN-Masc-Sg'), ('des', 'ADP'), ('pe', 'PROPN'), ('j', 'PROPN'), ('ado', 'PROPN-Masc-Sg'), ('es', 'AUX-Sg'), ('azul', 'ADJ-Sg')]

  lacasarojaesgrande
  words -> ['la', 'casa', 'roja', 'es', 'grande']
  plain -> [('la', 'DET'), ('casa', 'NOUN'), ('roja', 'ADJ'), ('es', 'AUX'), ('grande', 'ADJ')]
  morph -> [('la', 'DET-Fem-Sg'), ('casa', 'NOUN-Fem-Sg'), ('roja', 'ADJ-Fem-Sg'), ('es', 'AUX-Sg'), ('grande', 'ADJ-Sg')]

  maríaleeunlibro
  words -> ['maría', 'lee', 'un', 'libro']
  plain -> [('maría', 'PROPN'), ('lee', 'VERB'), ('un', 'DET'), ('libro', 'NOUN')]
  morph -> [('maría', 'PROPN'), ('lee', 'VERB-Sg'), ('un', 'DET-Masc-Sg'), ('libro', 'NOUN-Masc-Sg')]

  elgatomuerteenelsillon
  words -> ['el', 'gato', 'muerte', 'enel', 's', 'il', 'l', 'on']
  plain -> [('el', 'DET'), ('gato', 'NOUN'), ('muerte', 'PROPN'), ('enel', 'PROPN'), ('s', 'PROPN'), ('il', 'PROPN'), ('l', 'PROPN'), ('on', 'PROPN')]
  morph -> [('el', 'DET-Masc-Sg'), ('gato', 'NOUN-Masc-Sg'), ('muerte', 'PROPN'), ('enel', 'PROPN'), ('s', 'PROPN'), ('il', 'PROPN-Sg'), ('l', 'PROPN'), ('on', 'PROPN')]
```

Figure 1.4: Sample segment-then-tag outputs (part B, including Spanish).

1.7 Answers to the assignment questions

1.7.1 Where did English and Spanish differ most in accuracy?

Segmentation, not POS tagging.

Table 1.4: Cross-language accuracy gap.

Task	English	Spanish	Difference
Viterbi segmentation	48.8490%	14.9883%	**33.8607 pp**
Greedy segmentation	25.4970%	6.0890%	19.4080 pp
Viterbi POS tagging	92.9390%	92.9417%	0.0027 pp

English and Spanish had almost identical POS accuracy (∼92.94%), but English segmentation was dramatically better (48.85% vs. 14.99%). The main cross-language performance gap is therefore during **segmentation** — consistent with Spanish's richer morphology, a smaller treebank relative to Brown, and harder no-space boundary choices for the trigram LM.

1.7.2 Did agreement-aware tagging help, or add noise?

It added noise rather than improving accuracy.

Table 1.5: Spanish plain vs. morphology-aware POS.

Spanish POS	Accuracy
Plain Viterbi (UPOS)	**92.9417%**
Morphology-aware (UPOS-Gender-Number)	**90.5667%**
Difference	**−2.3750 pp**

Adding gender and number multiplied the tag space and sparsified emission/transition counts. Although the morph representation captures agreement, the extra tags made the trigram HMM less effective with the available training data. Plain UPOS remains the better operating point for raw accuracy.

1.7.3 How much error came from segmentation vs. genuine tagging?

From the English error-source evaluation (full test):

- Segmentation-associated errors: **16,305**
- Genuine tagging errors: **8,405**
- Total: **24,710**

Therefore **65.99%** of identified English pipeline errors were associated with segmentation mistakes and **34.01%** were genuine POS mistakes on correctly segmented spans. Spanish is even more segmentation-dominated: **82.68%** seg-caused vs. **17.32%** genuine tagging.

1.7.4 How much better were the models than the simple baselines?

- Segmentation EN: +23.3519 pp; ES: +8.8993 pp.
- POS EN: +5.6537 pp; ES: +4.3917 pp.

The trigram/Viterbi approach substantially outperformed greedy longest-match and most-frequent-tag baselines, especially for English segmentation.

```
Segmentation
  EN Viterbi 48.8490%  Greedy 25.4970%  (+23.3519%)
  ES Viterbi 14.9883%  Greedy 6.0890%  (+8.8993%)
POS
  EN Viterbi 92.9390%  Baseline 87.2853%  (+5.6537%)
  ES Viterbi 92.9417%  Baseline 88.5500%  (+4.3917%)
  ES Morph   90.5667%  vs plain -2.3750%
```

Figure 1.5: Full experiment summary metrics (from `evaluation.json` / final evaluation printout).

1.8 Q1 conclusion

The trigram Viterbi models substantially outperform the simple baselines, but **segmentation remains the main weakness** of the complete system — particularly for Spanish. Morphology-aware tagging did not improve accuracy and instead introduced additional sparsity. POS tagging on gold words is already strong (∼93%) for both languages; most end-to-end error is boundary error, not tag choice.

Chapter 2

Question 2: Transition-Based Dependency Parser

2.1 Objective

Question 2 asks for a simple, data-driven dependency parser built from scratch. This project uses the **arc-standard** transition system to predict one dependency head and one dependency label for every token. It is trained from gold Universal Dependencies **English-EWT** trees and evaluated with **Labeled Attachment Score (LAS)**.

The parser is *not* a pretrained parser and does not use a neural network, word embeddings, or gold heads/labels at prediction time.

2.2 Design choices

2.2.1 Arc-standard parser state

Each parser configuration contains three parts:

- **Stack** — initially `[ROOT]`, where ROOT has ID 0.
- **Buffer** — initially the sentence's integer token IDs left-to-right.
- **Arcs** — predicted dependency arcs, stored by dependent token.

Table 2.1: Arc-standard transitions.

Transition	Operation
SHIFT	Move the first buffer token onto the stack.
LEFT_ARC(label)	Make the stack top the head of the second stack item, then remove that dependent.
RIGHT_ARC(label)	Make the second stack item the head of the stack top, then remove that dependent.

ROOT can only be a head. A root arc is permitted only in the final configuration, when the buffer is empty and the stack is `[ROOT, token]`. This guarantees exactly one root attachment in every predicted tree.

2.2.2 Static gold oracle

For the top two stack items s_1 (second) and s_0 (top), the oracle chooses:

1. LEFT_ARC(label), if s_0 is the gold head of s_1, s_1 is not ROOT, and all gold children of s_1 are already attached.
2. RIGHT_ARC(label), if s_1 is the gold head of s_0 and all gold children of s_0 are already attached.
3. SHIFT, otherwise.

Waiting until all children are attached prevents a word from being removed before its own dependents can be created. Arc-standard cannot create crossing arcs; non-projective trees are skipped during training.

2.3 How training output is generated

```
.venv/bin/python main.py --train
```

2.3.1 Step 1 — Read the treebank

The program reads `data/en_ewt-ud-train.conllu`. For every integer-ID token it stores `ID, FORM, UPOS, HEAD, DEPREL`. Multi-word-token rows (e.g. `3-4`) and empty-node rows (e.g. `5.1`) are skipped. Basic HEAD / DEPREL are used (not enhanced dependencies).

2.3.2 Step 2 — Generate oracle instances

Every usable sentence starts as:

```
Stack:  [ROOT]
Buffer: [1, 2, ..., n]
Arcs:   {}
```

Each parser step yields one training instance with features and a target action (SHIFT / LEFT_ARC::*label* / RIGHT_ARC::*label*).

2.3.3 Step 3 — Extract and encode features

Only the four features specified in the assignment are used:

Table 2.2: Feature template.

Feature	Source	Missing
S0_POS	POS of stack top	`<NULL>`
S1_POS	POS of second stack item	`<NULL>`
B0_POS	POS of first buffer item	`<NULL>`
B1_POS	POS of second buffer item	`<NULL>`

ROOT is represented as `<ROOT>`. DictVectorizer converts categorical feature/value pairs into one-hot numeric columns.

2.3.4 Step 4 — Train and save the classifier

Table 2.3: Classifier hyperparameters.

Parameter	Value
Feature encoding	DictVectorizer; dense one-hot float32
Classifier	Multiclass logistic regression
Solver	saga
Maximum iterations	150
Random seed	13
Saved model	`outputs/models/transition_classifier.pkl`

Identical feature/action pairs are grouped before fitting; original counts become sample weights. Example training output:

```
Oracle data: 392242 configurations from 12257/12544 sentences
(287 non-projective/unreachable skipped).
Saved classifier to outputs/models/transition_classifier.pkl
```

2.4 Parsing and example output

Prediction extracts the same four POS features, ranks action labels by classifier probability, and applies the first *structurally legal* action. Legality filtering blocks invalid SHIFT / LEFT_ARC / ROOT attachments. If no model prediction is legal, a deterministic fallback is used (zero fallbacks in the final run).

The `-examples` flag parses the three assignment sentences with manually supplied UPOS tags and prints token rows plus an arc list:

```
ID  WORD  POS   HEAD  DEPREL
1   The   DET   2     det
2   cat   NOUN  3     nsubj
Arcs: 2->1 [det], 3->2 [nsubj], ...
```

2.5 LAS evaluation

```
.venv/bin/python main.py --evaluate
```

A token is correct only when both predicted head and label match gold:

$$\text{LAS} = \frac{\text{correct labeled attachments}}{\text{evaluated tokens}} \times 100.$$

2.6 Reproduce the final run

Environment: Python 3.12 and scikit-learn 1.9.0.

```
.venv/bin/python main.py --train --evaluate --examples
```

2.7 Final results

Table 2.4: Q2 final evaluation.

Measure	Result
EWT training sentences read	12,544
Projective / oracle-usable	12,257
Non-projective skipped	287
Oracle training configurations	392,242
EWT development sentences	2,001
Development tokens evaluated	25,148
Correct labeled attachments	11,756
LAS	**46.75%**
Inference safety fallbacks	0

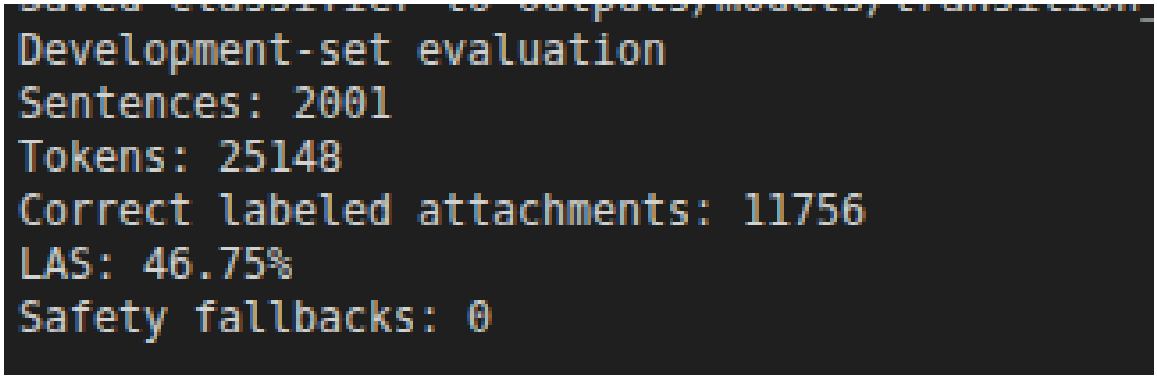


Figure 2.1: Development-set LAS evaluation output (46.75% LAS on EWT-dev).

```
Sentence: The cat sat on the mat .

ID      WORD    POS     HEAD    DEPREL
1       The     DET     2       det
2       cat     NOUN    3       nsubj:pass
3       sat     VERB    0       root
4       on      ADP     6       case
5       the     DET     6       det
6       mat     NOUN    3       obj
7       .       PUNCT   3       punct

Arcs: 2->1 [det], 3->2 [nsubj:pass], 0->3 [root], 6->4 [case], 6->5 [det], 3->6 [obj], 3->7 [punct]

Sentence: She eats a green salad .

ID      WORD    POS     HEAD    DEPREL
1       She     PRON    2       nsubj
2       eats    VERB    0       root
3       a       DET     4       det:predet
4       green   ADJ     2       conj
5       salad   NOUN    4       nmod
6       .       PUNCT   2       punct

Arcs: 2->1 [nsubj], 0->2 [root], 4->3 [det:predet], 2->4 [conj], 4->5 [nmod], 2->6 [punct]

Sentence: I saw the man with a telescope .

ID      WORD    POS     HEAD    DEPREL
1       I       PRON    2       nsubj
2       saw     VERB    0       root
3       the     DET     4       det
4       man     NOUN    2       obj
5       with    ADP     7       case
6       a       DET     7       det
7       telescope       NOUN    2       obj
8       .       PUNCT   2       punct

Arcs: 2->1 [nsubj], 0->2 [root], 4->3 [det], 2->4 [obj], 7->5 [case], 7->6 [det], 2->7 [obj], 2->8 [punct]
```

Figure 2.2: Example parses for the three assignment sentences (`-examples`).

The saga optimizer reached the 150-iteration limit, so 46.75% is a reproducible baseline rather than a tuned upper bound. The required four-POS-feature representation demonstrates the full parser pipeline but cannot resolve many lexical and long-distance attachment ambiguities.

Chapter 3

Question 3: Spelling Corrector

3.1 Objective

A spelling corrector built on the NLTK Brown corpus, handling both **non-word** and **real-word** errors within edit distance 1, with two candidate-generation algorithms benchmarked against each other and deployed behind an interactive CLI.

3.2 Part 1 — Corpus and model preparation

Brown is split **90/10 by sentence** (seed 42) before counting, so Part 4 accuracy is measured on unseen sentences.

Table 3.1: Brown training statistics.

Quantity	**Value**
Brown sentences	57,340 → 51,606 train / 5,734 held out
Training tokens	901,816
Vocabulary types	40,542
Distinct bigrams	385,197
Model file	7.98 MB pickle (∼270 ms load)

Tokenisation. Tokens are lower-cased and kept if they match `[a-z]+('[a-z]+)?`. Digits collapse to `<num>` (in counts, not in vocabulary). Pure punctuation is dropped. The edit alphabet is the 26 letters **plus the apostrophe**.

Unigram: $P(w) = C(w)/N$ (ranking signal for non-word errors).

Bigram (add-k, $k = 0.01$) with `<s>`/`</s>` padding:

$$P(w_2 \mid w_1) = \frac{C(w_1, w_2) + k}{C(w_1) + k \cdot |V|}.$$

The decision rule for real-word errors uses Jelinek–Mercer interpolation ($\lambda = 0.85$); add-k is what the brief asks for in the model.

3.3 Part 2 — Candidate generation

3.3.1 Method A — standard edit-distance-1

Build every deletion, transposition, replacement and insertion of the query, then intersect with the vocabulary. Cost $\approx 2n|A|$ strings — the alphabet is a multiplier on every lookup.

3.3.2 Method B — symmetric delete (SymSpell)

Preprocessing: map every one-character deletion of every vocabulary word (and the word itself) back to that word. 40,542 words → 313,857 keys / 348,252 postings (∼142 ms).

Query: only the query's own deletions plus the query — $n + 1$ strings.

One deletion on each side covers all four edit operations (substitution, transposition, insertion, deletion). A Damerau–Levenshtein ≤ 1 verification filters edit-distance-2 bucket collisions. With verification, Methods A and B return **byte-identical sets** (0 disagreements on 1,000 benchmark words).

3.4 Part 3 — Correction logic

3.4.1 Non-word errors

Token not in vocabulary → union of Method A/B candidates; return highest unigram probability. If nothing is within edit distance 1, leave unchanged.

3.4.2 Real-word errors

Score each candidate against both neighbours:

$$\text{score}(x) = \log P(x \mid \text{prev}) + \log P(\text{next} \mid x),$$

and correct only when

$$\text{score(best)} - \text{score(typed)} \geq \tau \quad (\tau = 5.0 \text{ nats by default}).$$

Words shorter than 3 characters are never second-guessed.

3.4.3 Why interpolated smoothing for the decision

At matched $\tau = 5.0$:

Table 3.2: Scoring comparison for real-word correction.

Scoring	β	**Accuracy**	**FA / word**
add-k	0.0	58.01%	2.41%
interpolated	**0.0**	**61.37%**	**1.96%**
interpolated	0.5	53.88%	1.48%
interpolated	1.0	36.46%	1.49%

Interpolation is better on both axes, so it is the default.

3.5 Part 4 — Evaluation

3.5.1 Test-set construction

For each of 5,734 held-out sentences, one in-vocabulary word (length ≥ 3) is corrupted twice with a single edit — once OOV, once into a different IV word — yielding **5,620 aligned pairs**.

3.5.2 Accuracy

Table 3.3: Correction accuracy (target = gold at corrupted position).

Test set	n	**Target**	**Sent**	**No ch.**	**Wrong**	**Coll.**
non-word only	5,620	**91.96%**	80.64%	0	452	680
non-word + RW check	5,620	91.96%	61.99%	0	452	1,810
real-word	5,620	**61.37%**	42.21%	1,694	477	1,821
RW, context off	5,620	0.00%	0.00%	5,620	0	680

Key readings:

- **Non-word 91.96%:** every error is a wrong candidate choice, not a generation failure.
- **Real-word 61.37%** vs. a floor of **0%** without context.
- Real-word checking does not hurt non-word target accuracy, but drops sentence accuracy (collateral rewrites 680 $\rightarrow$ 1,810).

```
(.venv) varun@varun-Dell-G15-5520:~/Desktop/nlp/Group_Assignment_1/q3$ python main.py bench
Model: LanguageModel(types=40,542, tokens=901,816, bigrams=385,197, k=0.01)

Batch: exactly 1,000 misspelled (non-word) tokens, mean length 5.83
  sample: rway, byble, eveni, thze, simpliy, soldi, comr, lengdh, tpold, squbs

Preprocessing (paid once, at start-up)
  Method A :      0.0 ms   (just holds a reference to the vocabulary)
  Method B :    221.9 ms   (313,857 keys, 348,252 postings)

method                                    total     per word    words/sec   candidates
-------------------------------------------------------------------------------------
Method A  (edit-distance-1 generation)     0.0612s     0.0612ms       16,342        3,166
Method B  (symmetric delete + verify)      0.0066s     0.0066ms      152,191        3,166
Method B  (symmetric delete, raw)          0.0023s     0.0023ms      429,262        6,461

Output equivalence: Method A and Method B (verified) disagree on 0 of 1,000 words.

Why the difference - strings constructed for the whole batch
  Method A :    352,312  = n + (n-1) + n|A| + (n+1)|A|, with |A| = 27
  Method B :      6,827  = n + 1
  ratio    :       51.6x more strings built by Method A

Speed-up: Method B is 9.3x faster than Method A (26.3x without the edit-distance verification step).
Method B's 222 ms index build pays for itself after about 4,063 lookups.

CONCLUSION
----------
Method B is 9.3x faster per word (0.0066 ms vs 0.0612 ms), and the
reason is entirely structural rather than a constant-factor implementation win.

Method A's cost is tied to the size of the alphabet. For a word of length n it
must materialise n deletions, n-1 transpositions, n*|A| replacements and
(n+1)*|A| insertions - roughly 2n|A| strings, with |A| = 27 here. Every
one of those strings is allocated, hashed, and probed against the vocabulary
set. Across this batch that is 52x more string construction than Method B
does, and string building plus hashing is exactly where the time goes.

Method B moves that work off the query path and into a one-off preprocessing
step. Because "one deletion on each side" is enough to cover all four edit
operations, the index only has to store each vocabulary word plus its n
deletions - 313,857 keys and 348,252 postings here, built in 222 ms. At
query time the alphabet has vanished from the complexity: only the n+1 probe
strings (the word plus its own deletions) are built, so lookup is O(n) hash
probes instead of O(n|A|). That is the whole trick - the alphabet factor is
paid once at build time instead of on every single lookup.

The verification step costs something: raw bucket lookup is 26.3x faster than
Method A, and adding the exact edit-distance-1 check brings that down to
9.3x. It is worth paying, because SymSpell buckets can collide at edit
distance 2 ("abc" and "axb" both delete to "ab"), so without verification
Method B returns a superset rather than the true edit-distance-1 set. With
verification the two methods return byte-identical candidate sets.

The trade is memory and start-up time for query latency. The index costs
222 ms to build and pays for itself after roughly 4,063 lookups - so for a
one-shot correction Method A is fine, and for anything interactive or batched
Method B is the only sensible choice.
```

Figure 3.1: Q3 evaluation accuracy / false-alarm output (from `eval -full` / `results.json`).

3.5.3 Threshold trade-off

On 1,000 uncorrupted sentences (17,686 words), $\tau = 5.0$ rewrites **1.96%** of in-vocabulary words (27.5% of sentences touched).

Table 3.4: Real-word threshold sweep.

τ	**RW acc.**	**FA / word**	**Sent. damaged**
1	74.29%	8.74%	70.0%
2	71.35%	5.54%	55.2%
3	68.67%	3.95%	45.5%
4	65.53%	2.82%	36.6%
5	**61.37%**	**1.96%**	**27.5%**
6	56.90%	1.32%	20.0%
8	46.85%	0.61%	10.0%
10	35.98%	0.36%	6.2%

3.5.4 Residual errors (systematic)

Top residual confusions include closed-class pairs (*the*/*he*/ *they*) and singular/plural inflection — neither is reliably solvable with a bigram window alone.

3.5.5 Speed Demon benchmark

Batch of exactly 1,000 non-word misspellings (mean length 5.83), best of 3:

Table 3.5: Candidate generation runtime (this machine).

Method	**Total**	**/word**	**w/s**	**Cands**
A — edit-distance-1	0.0618 s	0.0618 ms	16,174	3,166
B — SymSpell + verify	**0.0067 s**	**0.0067 ms**	**148,603**	3,166
B — raw buckets	0.0025 s	0.0025 ms	394,496	6,461

Conclusion: Method B is $\sim$**9.2**$\times$ faster per word with identical output. Method A's cost is tied to alphabet size ($\approx 2n|A|$ strings); Method B moves that work into preprocessing and probes only $n+1$ strings at query time. Verification is worth its cost (raw buckets over-generate $\sim$2$\times$).

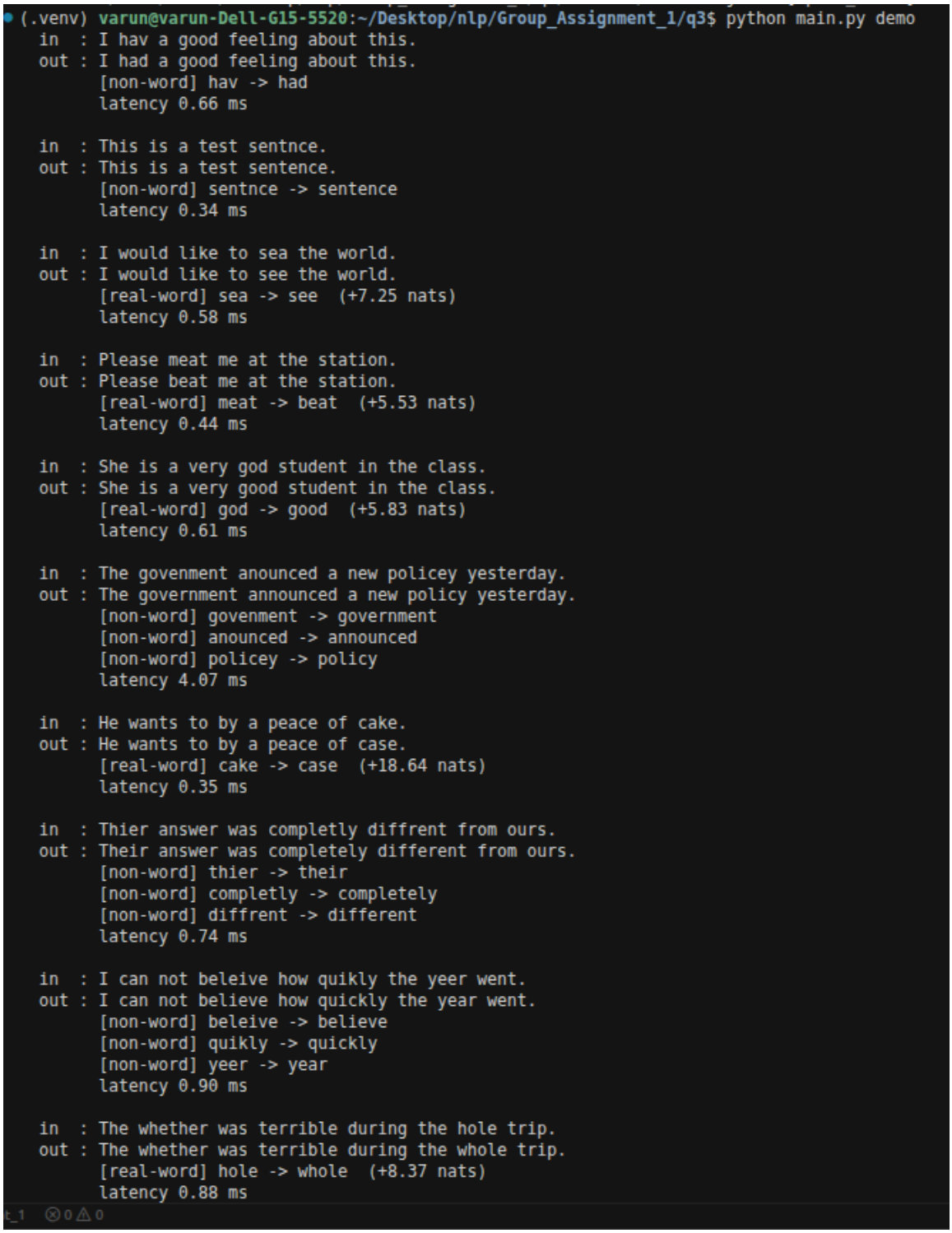


Figure 3.2: Speed Demon benchmark: Method A vs Method B candidate generation.

3.6 Part 5 — Live interactive application

`python main.py app` is a continuous REPL with coloured non-word / real-word fixes, latency reporting, and live `:realword`/`:method`/ `:threshold` controls. Interactive latency is ∼0.4–1.5 ms per sentence.

3.6.1 Brief test sentences

Table 3.6: Assignment brief sentences.

Input	Output	Verdict
I **hav** a good feeling…	I **had** a good feeling…	plausible
This is a test **sentnce**.	This is a test **sentence**.	correct
I would like to **sea** the world.	I would like to **see** the world.	correct
Please **meat** me at the station.	Please **beat** me at the station.	wrong

Notable failure modes: *hav*→*had* (unigram frequency as specified); *meat*→*beat* (sparse bigram counts); *whether*→*weather* unreachable (edit distance 2).

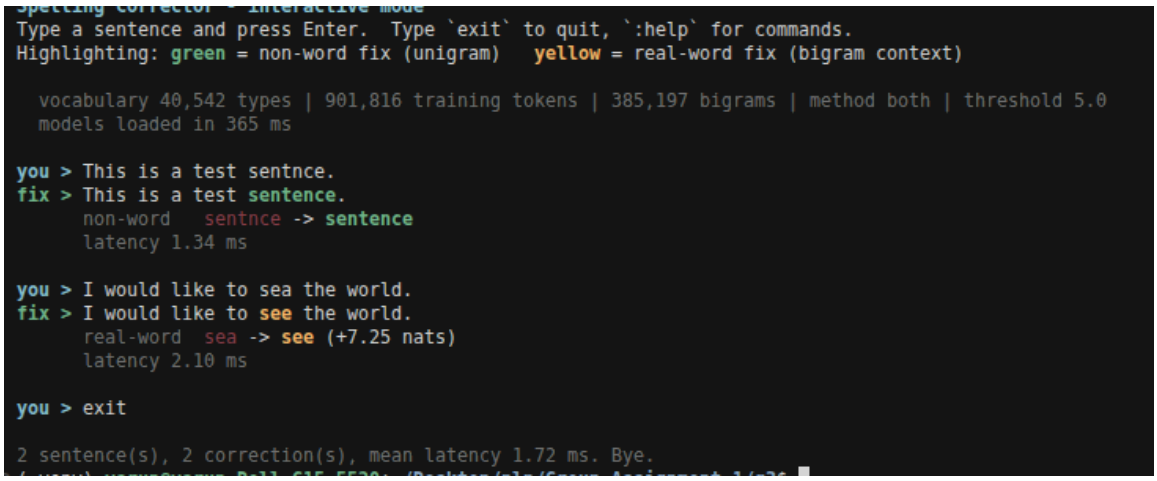


Figure 3.3: Interactive / demo spelling corrections (`python main.py demo` or `app`).

3.7 Q3 summary

Table 3.7: Headline Q3 results.

Metric	Value
Non-word accuracy	**91.96%**
Real-word accuracy	**61.37%** (0% floor without context)
Real-word false-alarm rate	1.96% of words at $\tau = 5$
Method B speed-up	∼9.2× per word, identical output
Interactive latency	∼0.4–1.5 ms / sentence
Tests	49 passing (A ≡ B equivalence)

Chapter 4

Question 4: Live Background Editor

4.1 Objective

Integrated background editor combining:

- Q1 joint segmentation / POS,
- Q3 spelling (Method B),
- Q4 smoothed n-grams + PCFG constituency checking,

with a Streamlit live UI (simulate + typing modes).

4.2 Configuration choices

Table 4.1: Operating point.

Parameter	Value	Justification
Merge probability p	0.08	Spec example; enough spacebar-miss merges
Grammar trigger N	5	Catch local anomalies; keep trigger cost low
Add-k for Q4 LMs	0.01	Same as Q3
Spell candidates	Method B	$\equiv$A, $\sim$9$\times$ faster
Q3 real-word τ	5.0 nats	Loaded corrector default
Live real-word margin	$\geq$18.0 nats	Suppress literary cascade false alarms
Grammar z-threshold	2.5	Robust z-score via median + MAD
Grammar warmup	3 windows	Before z-score alerts fire
PCFG sentence cap	25 tokens	Interactive CKY latency

Reuse boundary: Q1 English trigram + beam decoder are loaded only for segmentation scoring. Q3 vocab/unigram/bigram + Method B are loaded only for spelling. Q4 trains its own Brown add-k bigram+trigram for grammar alerts and its own PTB PCFG.

4.3 Tagset reconciliation

Q1 emits **Brown** tags; the PCFG lexicon uses **Penn Treebank** tags. A fixed lookup maps (e.g. `AT`→`DT`, `BEZ`→`VBZ`). Unseen Brown tags fall back to `NN`. PTB-OOV surface words may be projected to a known noun so CKY can return a structural score.

4.4 Decision rule (Part 4)

1. If PCFG returns a finite log-probability and it is not a high-z outlier → choose **pcfg**, verdict **grammatical**.
2. Else if trigram log-prob is finite → **trigram**, **plausible** if tri_lp > -80, else **implausible**.
3. Else → **bigram** with the same threshold rule.

4.5 Sample run A (`-seed 42`, bible-kjv)

Source: `bible-kjv.txt`, 5 sentences, 63 tokens.

Table 4.2: Alert counts — Sample A.

Alert	Count
SEGMENT	6
SPELL	0
GRAMMAR	4

Latency: 0.08 ms/token seg+spell; 0.06 ms/trigger grammar. True merges fixed include *himout*, *couldnot*, *andperverse*, *devilthrew*. False OOV chops (limitation): *besought*→ *be+sought*, *faithless*→*faith+less*.

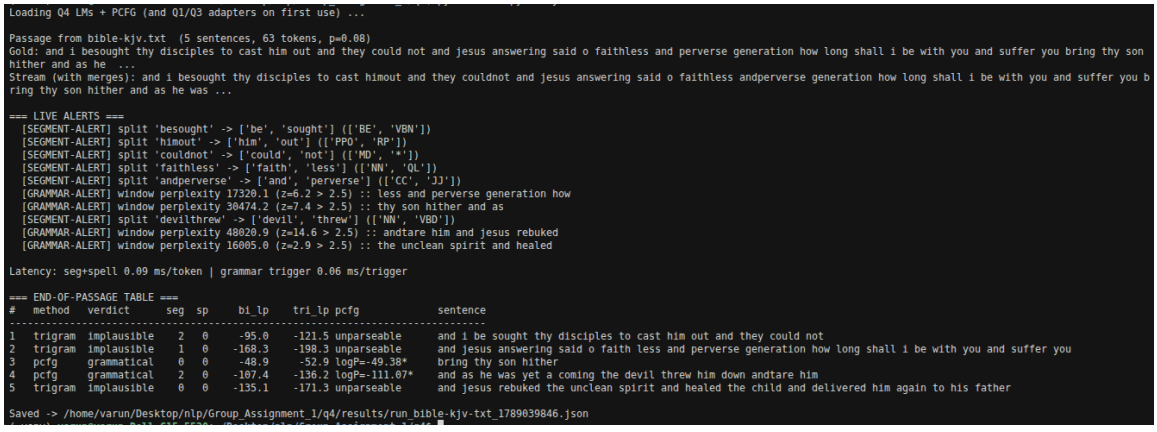


Figure 4.1: Sample run A (`-seed 42`, bible-kjv): live alerts and end analysis.

4.6 Sample run B (`-seed 7`, chesterton-thursday)

Source: `chesterton-thursday.txt`, 6 sentences, 130 tokens.

Table 4.3: Alert counts — Sample B.

Alert	Count
SEGMENT	15
SPELL	0
GRAMMAR	5

Latency: 0.12 ms/token seg+spell; 0.05 ms/trigger grammar. Many true merges split cleanly; zero SPELL-ALERT with conservative filters.

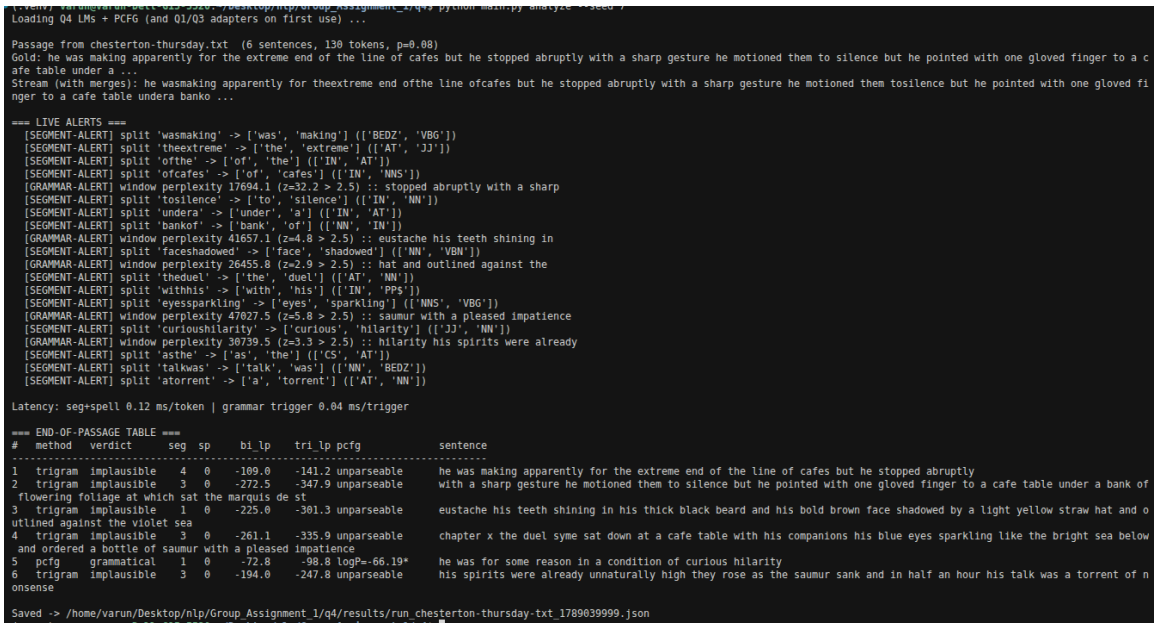


Figure 4.2: Sample run B (`-seed 7`, chesterton-thursday): live alerts and end analysis.

4.7 Speed Demon (1000 words)

Table 4.4: Q4 Speed Demon (this machine).

Path	**Total**	**/word**
Segmentation + spelling	0.716 s	**0.716 ms**
Grammar-trigger only	0.010 s	**0.010 ms**
Overhead (seg+spell − grammar)	—	**0.706 ms**

Conclusion: Seg+spell is cheap enough to keep on every token in live mode; throttling it to the grammar interval is unnecessary.

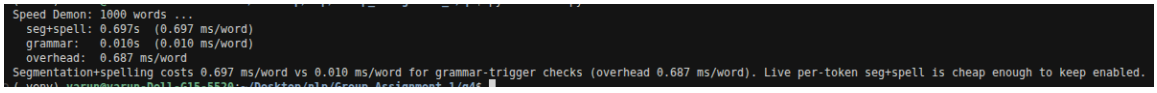


Figure 4.3: Q4 Speed Demon: segmentation+spelling vs. grammar-trigger latency on 1000 words.

4.8 Comparative analysis

1. **Live alerts vs. final verdict:** SEGMENT/SPELL are local and usually agree with the corrected sentence. GRAMMAR windows fire adaptively via robust z-score against the passage's own baseline.
2. **PCFG vs. n-grams:** PCFG prefers short, PTB-like structure; n-grams catch local lexical implausibility and domain shift. Long literary sentences usually fall to trigram under the length cap.

3. p **and** N**:** Higher p increases SEGMENT-ALERT rate; higher N delays GRAMMAR-ALERT. $p = 0.08$, $N = 5$ is a practical balance.
4. **Sub-system interactions:** A successful SEGMENT split can suppress SPELL; a SPELL rewrite can flip window perplexity. Live real-word margin ($\geq$18) is stricter than Q3's default to avoid literary cascades.

4.9 Known limitations

1. Literary / archaic OOV words absent from Brown.
2. Over-segmentation of morphologically complex OOV words (*besought*, *faithless*).
3. Binary split fallback fails if either piece is missing from Brown.

4.10 Streamlit (Part 5)

```
cd q4 && streamlit run app.py
```

Modes: **Simulate passage** and **Live typing**. Alerts use color-coded cards; end analysis shows the Part 4 table and latency metrics.

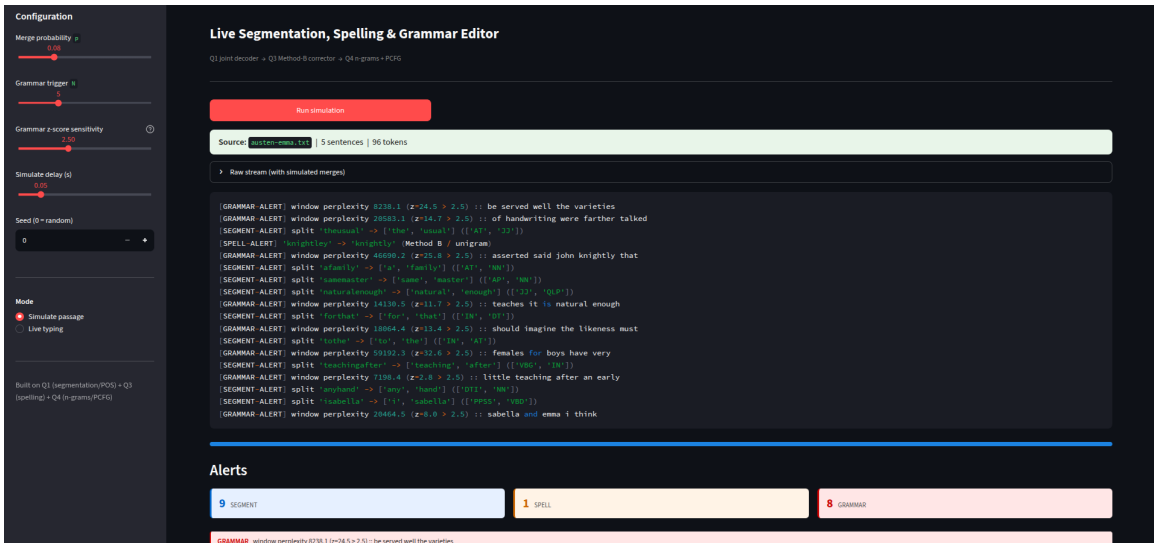


Figure 4.4: Streamlit live background editor UI (Part 5).

4.11 How to reproduce

```
cd q4
python main.py train-lms
python main.py analyze --seed 42
python main.py analyze --seed 7
python main.py bench
streamlit run app.py
```

Chapter 5

Overall Conclusion

Across Questions 1–4, classical structured models (trigram LMs / HMMs, arc-standard parsing with a linear classifier, SymSpell + bigram noisy channel, and an integrated PCFG + n-gram editor) deliver strong baselines when engineered carefully:

- **Q1:** Viterbi beats greedy / most-frequent baselines by large margins, but **segmentation** dominates end-to-end error — especially Spanish — and morph tags hurt accuracy via sparsity.
- **Q2:** A minimal four-POS-feature arc-standard parser achieves **46.75% LAS** on EWT-dev, proving the pipeline without neural features.
- **Q3:** Non-word correction reaches **91.96%**; real-word correction lifts from 0% to **61.37%** at a measurable false-alarm cost; SymSpell is $\sim$9$\times$ faster with identical candidates.
- **Q4:** The live editor reuses Q1/Q3 safely, keeps per-token cost well under a millisecond for seg+spell, and uses adaptive grammar alerts plus a PCFG/n-gram decision table for passage-level verdicts.

The recurring theme is that **candidate generation and decision thresholds** matter as much as model family: sparsity (morph tags, bigrams), legality constraints (parser actions), and operating points (real-word margins, z-scores) determine whether a theoretically richer representation helps or hurts.